

---

## REQUIREMENTS NOT MET

---

None

---

## PROBLEMS ENCOUNTERED

---

None Encountered

---

## FUTURE WORK/APPLICATIONS

---

In this lab, we learn how to do interrupts from an input! This allows us to have simultaneous actions within our loop with our interrupt firing as necessary; leading to much more efficient and modular programming! This further allows us to make more advanced machines and programs for the microprocessor, maybe a controller for a TV or a key fob!



---

## PRE-LAB EXERCISES

---

- i. Assuming that no interrupt has been previously configured, devise and describe a generalized series of steps for configuring any interrupt within the ATxmega128A1U, i.e., not just an interrupt within the TC system. Explain what happens in hardware (in other words, without the programmer's intervention) when the processor detects and then services (and returns from) an interrupt. Be as specific as possible, referencing certain registers when appropriate. You can assume that the `reti` instruction does not count as programmer intervention. You may provide a flowchart as a response, if desired.

To configure any interrupt on the ATxmega128A1U, the programmer must first initialize the Stack Pointer (SP) so the hardware can save the return address on the data stack. Every interrupt requires a defined hardware trigger and an assigned priority level.

For timer triggers, this involves setting the period register (`TCXn_PER`), selecting a clock (`TC_CLKSEL_DIVn_gc`), and enabling the overflow level in `TCXn_INTCTRLA`, which causes the hardware to set the overflow interrupt flag (`OVFIF`) in `TCXn_INTFLAGS` upon hitting the period value.

For external inputs, we must initialize the relevant GPIO, along with the Input/Sense Configuration bits (`ISC[2:0]`) in `PORTX_PINnCTRL` which are configured to detect edge transitions, which automatically sets an interrupt flag (`INT0IF` or `INT1IF`) in `PORTX.INTFLAGS` when triggered.

Once the chosen level is enabled in `PMIC.CTRL` and interrupts are globally active with the `sei` instruction, a valid trigger causes the CPU to pause execution, push the Program Counter onto the stack, and vector to the interrupt address (e.g., `TCC0_OVF_vect`). Upon an interrupt occurring the `SREG` enables the global interrupt flag.

Because each vector is only four bytes wide, it simply contains an `RJMP` instruction to the main interrupt program. While the hardware automatically clears `OVFIF` when vectoring to a timers interrupt, the programmer must explicitly clear the port flags by writing 1 to `PORTX.INTFLAGS` inside a ports interrupt to prevent immediate re-triggering upon exit. Once the interrupt is fully exited using `reti`, the `SREG` disables the global interrupt flag.

---

## PSEUDOCODE/FLOWCHARTS

---

### LAB3\_1:

.include "ATxmega128A1Udef.inc"

.org 0

.rjmp MAIN

.org interruptVector

rjmp INTERRUPT

.org 0x0100

MAIN:

setup Stack Pointer

rcall IO initialization

rcall TC initialization

rcall PMIC initialization

sei

LOOP:

rjmp LOOP

DONE:

rjmp DONE

; SUBROUTINES & INTERRUPT SERVICE:

IO:

push registers

load 1 to registers

store register to PORTC Direction

pop registers

return

TC:

push registers

load TCC0\_PER with calculated period for 47ms

load TCC0\_CTRLA with the correct prescaler & TC\_OVFINTLVL (LO/MED/HI)

pop registers

return

PMIC:

push registers

enable relevant level interrupts in PMIC\_CTRL (PMIC\_(LO/MED/HI)LVLEN\_bm)

pop Registers

return

INTERRUPT:

push registers

load SREG

push SREG

load 1 to PORTC\_OUTTGL

pop SREG

store SREG

pop registers

return

**LAB3\_2a:**

.include "ATxmega128A1Udef.inc"

.org 0

.rjmp MAIN

.org interruptVector

rjmp INTERRUPT

.org 0x0100

MAIN:

setup Stack Pointer

rcall IO initialization

rcall PMIC initialization

sei

LOOP:

rjmp LOOP

DONE:

rjmp DONE

; SUBROUTINES & INTERRUPT SERVICE:

IO:

push registers

load 0b000000100

store to PORTF\_DIRCLR ; Set S1 on the SLB to input (PF2)

load 0xFF

store to PORTC\_DIRSET ; Set all of PORTC to output

invert all outputs to PORTC

load 0x02 to ISC (lower three bits of PORTF\_PIN2CTRL) ; To trigger on falling edge (active-low switch)

load (1<<2) to PORTF\_INT0MASK ; to use INT0 flag as our interrupt

load 1 to PORTF\_INTCTRL ; (INT0 = [1:0], 0 = OFF, 1 = LO, 2 = MED, 3 = HI)

pop registers

return

PMIC:

push registers

load PMIC\_LOLVLEN\_bm to PMIC\_CTRL

pop Registers

return

INTERRUPT:

push registers

load & Push SREG

use a dedicated register for counting (R20)

inc r20

cpi r20, 255

brne NO\_RESET

load 0 to r20 ; reset r20

NO\_RESET:

sts r20, PORTC\_OUT

clear interrupt flag

pop & store SREG

pop registers

reti

### LAB3\_2b:

```
.include "ATxmega128A1Udef.inc"
```

```
.org 0
```

```
rjmp MAIN
```

```
.org PORTF_INT0_vect ; Vector for S1 switch (Debounce start)
```

```
rjmp DEBOUNCE_INTERRUPT
```

```
.org TCF0_OVF_vect ; Vector for Debounce Timer (Increment)
```

```
rjmp INC_INTERRUPT
```

```
.org 0x0100
```

```
MAIN:
```

```
setup Stack Pointer
```

```
rcall IO_INIT
```

```
rcall POLL_TC_INIT    ; Initialize 47ms polling timer
```

```
rcall DEBOUNCE_TC_INIT ; Initialize stopped 40ms debounce timer
```

```
rcall PMIC_INIT
```

```
sei
```

```
LOOP:
```

```
read TCD0_INTFLAGS
```

```
if OVF flag is NOT set, rjmp LOOP
```

```
toggle PD6 (Blue LED)
```

```
clear OVF flag
```

```
rjmp LOOP
```

```
DONE:
```

```
rjmp DONE
```



; SUBROUTINES:

IO\_INIT:

push registers

set PF2 to input

set PORTC to output

set PD6 to output

invert pins 0-7 of PORTC

set PF2 pin control for falling-edge detection

set PORTF\_INT0MASK to include PF2

set PORTF\_INTCTRL to enable Low-Level interrupts

pop registers

return

POLL\_TC\_INIT:

push registers

load 47ms period into TCD0

set TCD0\_CTRLA to enable timer (div8)

pop registers

return

DEBOUNCE\_TC\_INIT:

push registers

load Period for 40ms into TCF0\_PER

set TCF0\_INTCTRLA to enable Low-Level interrupts

set TCF0\_CTRLA to OFF

pop registers

return

PMIC\_INIT:

push registers

load PMIC\_LOLVLEN\_bm into PMIC\_CTRL

pop registers

return

; INTERRUPT SERVICES

DEBOUNCE\_INTERRUPT:

push registers, Push SREG

disable PORTF interrupts (to lock out switch)

start TCF0 timer

pop SREG, Pop registers

reti

INC\_INTERRUPT:

push registers, Push SREG

stop TCF0 timer

reset TCF0 count

clear TCF0\_INTFLAGS

if PF2 is high,

rjmp SKIP\_INC (active-low, noise)

inc r20

cpi r20, 256

brne NO\_RESET

load 0 to r20

NO\_RESET:

store r20 to PORTC\_OUT

SKIP\_INC:

clear PORTF\_INTFLAGS

enable PORTF interrupts

pop SREG, Pop registers

reti

---

## PROGRAM CODE

---

### Lab3\_1:

```
/*
Lab3.asm
Created: 6/4/2026 9:46:41 AM
Author : Evan Baesler

ABSTRACT: We are utilizing timers to cause an overflow flag every 47ms
          which will trigger an interrupt, strobing an LED (toggling GPIO.)

OBJECTIVES: (1) Initialize Stack Pointer at SRAM end.
              (2) Determine IO direction (singular pin)
              (2) Start timer with a 47ms loop
              (3) Check for OVIF flag, and rjmp to an interrupt when checked
              (4) Clear flag
*/

.include "ATxmega128A1Udef.inc"

.org 0
rjmp MAIN

.org TCC0_OVF_vect
rjmp INTERRUPT

.org 0x0100
MAIN:

ldi r16, low(RAMEND)
sts CPU_SPL, r16
ldi r16, high(RAMEND)
sts CPU_SPH, r16
; Stack Pointer initialized, we can use subroutines for
; IO/TC initialization!

rcall IO_INIT
rcall TC_INIT
rcall PMIC_INIT
sei

LOOP:

rjmp LOOP

DONE:
rjmp DONE

; -----
; SUBROUTINES
; -----

IO_INIT:
push r16

ldi r16, 0x01
```

```
sts PORTC_DIRSET, r16 ; PORTC_DIR -> 0x01 -> Output on PC0

pop r16
ret

TC_INIT:
push r16

; SCF = 2MHz, PRE = 8, D = 47*10^-3, PER = 11750 Ticks
ldi r16, low(12050)
sts TCC0_PER, r16
ldi r16, high(12050)
sts TCC0_PER+1, r16 ; Set period of 47ms

ldi r16, TC_CLKSEL_DIV8_gc
sts TCC0_CTRLA, r16 ; Start counting with div8 prescaler

ldi r16, TC_OVFINTLVL_LO_gc
sts TCC0_INTCTRLA, r16 ; Set the TC to low priority interrupt

pop r16
ret

PMIC_INIT:
push r16

; Enable Low-Level Interrupts in the PMIC
ldi r16, PMIC_LOLVLEN_bm
sts PMIC_CTRL, r16

pop r16
ret
; -----
; INTERRUPT
; -----

INTERRUPT:
push r16
lds r17, CPU_SREG
push r17

ldi r16, 0x01
sts PORTC_OUTTGL, r16

pop r17
sts CPU_SREG, r17
pop r16
reti
```

## Lab3\_2a:

```
/*
Lab3.asm
Created: 6/7/2026 3:02:45 PM
Author : Evan Baesler

ABSTRACT: We are utilizing S1 on the 00TB SLB with interrupts to do a binary
          counter from 0 - 255, and then resetting back down to 0.

OBJECTIVES: (1) Initialize Stack Pointer at SRAM end.
              (2) Utilize subroutines to initialize IO & PMIC
              (3) Clear counter register (r20)
              (4) IO: Set PF2 in, PORTC out, invert PORTC, connect PF2 to LOLVL INT0
              (5) PMIC: Enable LOLVL interrupts
              (6) sei to do global interrupt enable
              (7) INTERRUPT: Push registers, push CPU_SREG to save flags, check if
                  r20 = 255, reset to 0 if true, increment r20 if false. Send value
                  to PORTC LEDs; clear interrupt flags, restore CPU_SREG to initial
                  value
*/

.include "ATxmega128A1Udef.inc"

.org 0
rjmp MAIN

.org PORTF_INT0_vect
rjmp INTERRUPT

.org 0x0100
MAIN:

ldi r16, low(RAMEND)
sts CPU_SPL, r16
ldi r16, high(RAMEND)
sts CPU_SPH, r16

clr r20 ; clear counter
rcall IO_INIT
rcall PMIC_INIT
sei

LOOP:
rjmp LOOP

DONE:
rjmp DONE

; SUBROUTINES

IO_INIT:
push r16
ldi r16, (1<<2) ; bit 2 for PORTF (PF2)
sts PORTF_DIRCLR, r16
ldi r16, 0xFF
sts PORTC_DIRSET, r16

ldi r16, PORT_INVEN_bm
sts PORTC_PIN0CTRL, r16
```

```
sts PORTC_PIN1CTRL, r16
sts PORTC_PIN2CTRL, r16
sts PORTC_PIN3CTRL, r16
sts PORTC_PIN4CTRL, r16
sts PORTC_PIN5CTRL, r16
sts PORTC_PIN6CTRL, r16
sts PORTC_PIN7CTRL, r16

lds r16, PORTF_PIN2CTRL ; we do not want to overwrite any defaults for the register
andi r16, 0b11111000 ; setting 010 to ISC[2:0], to create falling-edge ctrl
ori r16, 0b00000010
sts PORTF_PIN2CTRL, r16

ldi r16, 0x01
sts PORTF_INTCTRL, r16 ; create low-level interrupt

ldi r16, (1<<2)
sts PORTF_INT0MASK, r16 ; sets pin 2 to interrupt 0 on PORTF

pop r16
ret

PMIC_INIT:
push r16
ldi r16, PMIC_LOLVLEN_bm
sts PMIC_CTRL, r16
pop r16
ret

; INTERRUPT
INTERRUPT:
push r16
lds r16, CPU_SREG
push r16

inc r20 ; dedicated counting register
cpi r20, 255
brne NO_RESET
clr r20 ; reset
NO_RESET:
sts PORTC_OUT, r20
ldi r16, 1
sts PORTF_INTFLAGS, r16 ; clear INT0IF
pop r16
sts CPU_SREG, r16
pop r16
reti
```

## Lab3\_2b:

```
/*
Lab3.asm
Created: 6/7/2026 3:56:44 PM
Author : Evan Baesler

ABSTRACT: We are utilizing S1 on the 00TB SLB with interrupts to do a binary
          counter from 0 - 255, and then resetting back down to 0; simultaneously
          we will strobe our blue LED on and off every 47ms. This differs from
          lab3_2a by adding a timer-based debounce to the switch S1.

OBJECTIVES: (1) Initialize Stack Pointer at SRAM end.
            (2) Utilize subroutines to initialize IO & PMIC
            (3) Clear counter register (r20)
            (4) IO: Set PF2 in, PORTC out, invert PORTC, connect PF2 to LOLVL INT0
            (5) PMIC: Enable LOLVL interrupts
            (6) sei to do global interrupt enable
            (7) INTERRUPT: Push registers, push CPU_SREG to save flags, check if
                    r20 = 255, reset to 0 if true, increment r20 if false. Send value
                    to PORTC LEDs; clear interrupt flags, restore CPU_SREG to initial
                    value. Utilize a debounce to ensure no double-firing.

*/

.include "ATxmega128A1Udef.inc"

.org 0
rjmp MAIN

.org PORTF_INT0_vect ; When PF2 is pressed, we start a debounce timer.
rjmp DEBOUNCE_INTERRUPT

.org TCF0_OVF_vect ; When the above timer reaches OVF, we increment.
rjmp INC_INTERRUPT

.org 0x0100
MAIN:

ldi r16, low(RAMEND) ; Stack Pointer Initialization
sts CPU_SPL, r16
ldi r16, high(RAMEND)
sts CPU_SPH, r16

clr r20 ; Clear Binary Counter

; Initialization Subroutines
rcall IO_INIT ; Enable PF2 as input, PD6 & PC out.
rcall POLL_TC_INIT ; Initialize 47ms TC for Blue LED PWM
rcall DEBOUNCE_TC_INIT ; Initialize a stopped 40ms debounce TC
rcall PMIC_INIT ; Enable low-level interrupts in PMIC
sei

LOOP:

lds r16, TCD0_INTFLAGS ; Read the polling timer interrupt flags
sbrs r16, 0 ; Skip the toggle if the OVF (bit 0) is NOT set
rjmp LOOP ; If flag is 0, keep looping

ldi r16, (1<<6) ; Bit 6 for PD6 (Blue LED)
sts PORTD_OUTTGL, r16 ; Toggle the blue LED
```

```
ldi r16, 1 ; clear OVIF by writing 1 to it
sts TCD0_INTFLAGS, r16

rjmp LOOP ; Back to polling

DONE:
rjmp DONE ; Never-ending loop

; SUBROUTINES

IO_INIT:
push r16
ldi r16, (1<<2) ; PF2 -> Input
sts PORTF_DIRCLR, r16
ldi r16, 0xFF ; PC -> Output
sts PORTC_DIRSET, r16
ldi r16, (1<<6) ; PD6 -> Output
sts PORTD_DIRSET, r16

ldi r16, PORT_INVEN_bm ; Invert active-low LED outputs
sts PORTC_PIN0CTRL, r16
sts PORTC_PIN1CTRL, r16
sts PORTC_PIN2CTRL, r16
sts PORTC_PIN3CTRL, r16
sts PORTC_PIN4CTRL, r16
sts PORTC_PIN5CTRL, r16
sts PORTC_PIN6CTRL, r16
sts PORTC_PIN7CTRL, r16

lds r16, PORTF_PIN2CTRL ; We do not want to overwrite any defaults for the register
andi r16, 0b11111000 ; Setting 010 to ISC[2:0], to create falling-edge trigger
ori r16, 0b00000010
sts PORTF_PIN2CTRL, r16

ldi r16, 0x01
sts PORTF_INTCTRL, r16 ; Set PF to a low-level interrupt

ldi r16, (1<<2)
sts PORTF_INT0MASK, r16 ; Allows PF2 to trigger INT0 flag

pop r16
ret

POLL_TC_INIT:
push r16
; Set the Timer Period to 11,750 cycles for a prescaler of 8, for 47ms period
ldi r16, low(11750)
sts TCD0_PER, r16
ldi r16, high(11750)
sts TCD0_PER+1, r16
ldi r16, TC_CLKSEL_DIV8_gc
sts TCD0_CTRLA, r16
pop r16
ret

DEBOUNCE_TC_INIT:
push r16
; Set the Timer Period to 10,000 cycles with a prescaler of 8, for a 40ms period
```



```
ldi r16, low(10000)
sts TCF0_PER, r16
ldi r16, high(10000)
sts TCF0_PER+1, r16

; Enable low-level interrupts for TCF0
ldi r16, TC_OVFINTLVL_LO_gc
sts TCF0_INTCTRLA, r16

ldi r16, TC_CLKSEL_OFF_gc ; Start in OFF position
sts TCF0_CTRLA, r16
pop r16
ret

PMIC_INIT:
push r16
ldi r16, PMIC_LOLVLEN_bm ; Enable low-level interrupts
sts PMIC_CTRL, r16
pop r16
ret

; INTERRUPT SERVICES
; Triggered upon PF2 press
DEBOUNCE_INTERRUPT:
push r16
lds r16, CPU_SREG ; Store SREG flags to push later
push r16

ldi r16, 0
sts PORTF_INTCTRL, r16 ; Disable any interrupts while in debounce loop

ldi r16, TC_CLKSEL_DIV8_gc
sts TCF0_CTRLA, r16 ; Start counting 40ms debounce

pop r16
sts CPU_SREG, r16
pop r16
reti

; Triggered upon debounce timer ending
INC_INTERRUPT:
push r16
lds r16, CPU_SREG
push r16

ldi r16, TC_CLKSEL_OFF_gc
sts TCF0_CTRLA, r16 ; Stop the timer clock

clr r16
sts TCF0_CNT, r16
sts TCF0_CNT+1, r16 ; Reset count registers

ldi r16, 1
sts TCF0_INTFLAGS, r16 ; Clear timer overflow flag (OVFIF)

; Check if switch is still low (valid press)
lds r16, PORTF_IN
sbrc r16, 2
rjmp SKIP_INC ; If pin is high, it's a false bounce.
```

```

cpi r20, 255 ;
inc r20 ; increment after compare to have true 0-255 on LEDs

brne NO_RESET
clr r20

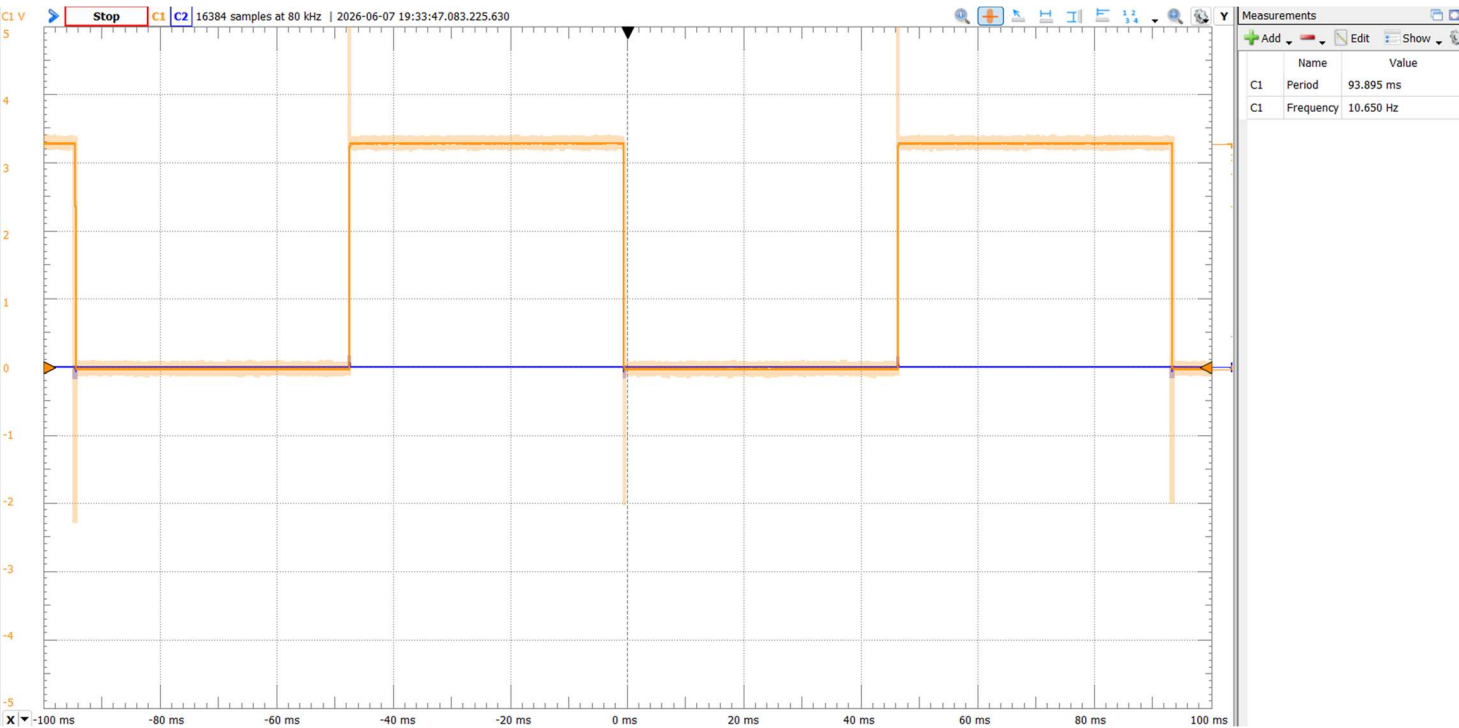
NO_RESET:
sts PORTC_OUT, r20 ; Store value to PORTC LEDs

SKIP_INC:
; Clear interrupt flag
ldi r16, 1
sts PORTF_INTFLAGS, r16

; Allow for interrupts again on PF
ldi r16, 1
sts PORTF_INTCTRL, r16

pop r16
sts CPU_SREG, r16
pop r16
reti
```

APPENDIX



| Word Address  |   |              |                                               |
|---------------|---|--------------|-----------------------------------------------|
| ATxmega128A1U |   | ATxmega64A1U |                                               |
| 0             |   | 0            | Application Section (bytes)<br>(128K/64K)     |
|               |   |              |                                               |
|               |   |              |                                               |
|               |   |              | ...                                           |
| FFFF          | / | 77FF         | Application Table Section (bytes) (8K/<br>4K) |
| F000          | / | 7800         |                                               |
| FFFF          | / | 7FFF         |                                               |
| 10000         | / | 8000         | Boot Section (bytes)<br>(8K/4K)               |
| 10FFF         | / | 87FF         |                                               |

Figure 2: Program Memory Addressing

| Byte Address | ATxmega64A1U                | Byte Address | ATxmega128A1U               |
|--------------|-----------------------------|--------------|-----------------------------|
| 0            | I/O Registers (4K)          | 0            | I/O Registers (4KB)         |
| FFF          |                             | FFF          |                             |
| 1000         | EEPROM (2K)                 | 1000         | EEPROM (2K)                 |
| 17FF         |                             | 17FF         |                             |
|              | RESERVED                    |              | RESERVED                    |
| 2000         | Internal SRAM (4K)          | 2000         | Internal SRAM (8K)          |
| 2FFF         |                             | 3FFF         |                             |
| 4000         | External Memory (0 to 16MB) | 3000         | External Memory (0 to 16MB) |
| FFFFFF       |                             | FFFFFF       |                             |

Figure 3: Data Memory Addressing